

Leakage, Duplicates and the Limits of Stacking: Reproducing and Improving a Stacking Ensemble for Heart-Disease Prediction

SIT307 Machine Learning — Task 11.1HD (Machine Learning Mini-Research)

Ashriya Singla (225682089) — 02 October 2026

Table 1. Submission links.

Deliverable	Link
Video presentation (≤ 5 min, unlisted)	[INSERT UNLISTED YOUTUBE LINK]
Code, data, notebook and README	https://github.com/ashriya-singla/SIT307-11.1HD

Abstract. Bhagat *et al.* [1] report that a 5-fold stacking ensemble of six classifiers predicts heart disease with 98.53% accuracy on the 1,025-record Kaggle heart-disease dataset. I reproduce the full study (six base learners, stacking, all reported metrics) and obtain 98.54% on an 80/20 split with seed 42 and $99.71\% \pm 0.89\%$ over 30 random splits, confirming that the headline number is reproducible. A data audit, however, shows that 723 of the 1025 rows are exact duplicates (only 302 unique patients), so on average 97.1% of test rows also appear in the training set. When the same pipeline is evaluated on unique patients its accuracy falls to 81.91%. I then propose **LF-DPS** (Leakage-Free, Diversity-Pruned Stacking), which combines patient-level de-duplication, repair of sentinel-coded missing values, feature-type-aware encoding, nested hyper-parameter optimisation and greedy selection of complementary base learners, evaluated with 5×5 repeated nested cross-validation and corrected resampled t-tests. On locked external patients, the paper's protocol over-estimates accuracy by 19.0 points, whereas LF-DPS's internal estimate is within 0.0 points of its true external accuracy, using a mean of 2.1 base learners instead of six.

1. Introduction

Cardiovascular disease causes about 17.9 million deaths per year, and early risk stratification from routinely collected clinical variables is a long-standing machine-learning application [1], [10], [11]. Numerous studies on the UCI/Kaggle heart-disease data report accuracies above 95% [1], [17], [18]. This is far higher than the roughly 77–85% typically reported on the original Cleveland cohort [11]. Such a gap should prompt scrutiny of how models were validated, because data leakage is one of the most common causes of irreproducible ML-based science [7], [8]. This report reviews and reproduces one such paper [1] (Part 1) and designs, implements and evaluates an improved learning pipeline that targets its key limitation (Part 2). Section 7 summarises the video presentation (Part 3).

2. Part 1 — Critical Review of the Selected Paper

2.1 Research problem, motivation and research gap

The paper addresses binary prediction of heart disease (target = 1) from 13 clinical attributes. Its motivation is that conventional risk scores (e.g., Framingham) predict individual risk poorly, while ML can exploit patterns in electronic health data [1]. The related-work table in [1] summarises state-of-the-art results ranging from 85.48% (majority-vote ensemble [12]) and 88.47% (HRFLM hybrid RF + linear model [2]) to 92.37% (FCMIM-SVM with LOSO validation [10]) and 97.89% (Relief + RF on merged datasets [9]). The gap the authors claim is that a heterogeneous stacking ensemble of six standard classifiers, with a meta-classifier trained on their predictions, can outperform both the individual classifiers and these prior methods. The claimed contributions are (i) a merged dataset from four sources, (ii) six classifiers, (iii) 5-fold stacking, and (iv) a comparative evaluation.

2.2 Dataset and feature set

The study uses the Kaggle "Heart Disease Dataset" [19], a 1,025-row file derived from the UCI Cleveland, Hungary, Switzerland and VA Long Beach databases [10]. It contains 13 predictors plus the target (Table 2). My audit (Table 3) reveals two properties that the paper does not report. First, only 302 rows are unique: every patient occurs 3–8 times, and no duplicate group has conflicting labels, so

the file is effectively the 302-patient Cleveland data replicated. Second, although the paper's own attribute table gives $ca \in [0, 3]$ and $thal \in [1, 3]$, the data contain $ca = 4$ (4 patients) and $thal = 0$ (2 patients). These counts match exactly the 4 and 2 missing ('?') values of the original Cleveland file [10], so they are disguised missing values rather than real categories. The paper states that it imputes missing values with linear regression, but there are no explicit missing values to impute.

Table 2. Feature set (13 predictors) used by the paper [1] and in this study.

Attribute	Description	Type (used in Part 2)
age, trestbps, chol, thalach, oldpeak	Age; resting BP; cholesterol; max heart rate; ST depression	Continuous
sex, fbs, exang	Sex; fasting blood sugar > 120 mg/dl; exercise-induced angina	Binary
cp, restecg, slope, thal	Chest-pain type; resting ECG; ST slope; thalassaemia	Nominal (one-hot)
ca	Number of major vessels coloured by fluoroscopy (0–3)	Ordinal count
target	1 = heart disease, 0 = no disease	Label

Table 3. Data audit of heart.csv (results/data_audit.json).

Property	Value
Rows / predictors	1025 / 13
Unique rows (patients)	302
Exact duplicate rows	723 (70.5%)
Copies per patient (count of patients)	3× : 187, 4× : 114, 8× : 1
Class balance, all rows (0 / 1)	499 / 526
Class balance, unique rows (0 / 1)	138 / 164
Explicit NaN values	0
Sentinel $ca = 4$ (rows / patients)	18 / 4
Sentinel $thal = 0$ (rows / patients)	7 / 2

2.3 Machine-learning methods and experimental protocol

The pipeline in [1] (Fig. 1 and Table 8 of the paper) is as follows. The data are split into training and testing sets and then pre-processed (linear-regression imputation, normalisation, categorical encoding). Six base classifiers are trained: Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), Extreme Gradient Boosting (XGB), Gaussian Naive Bayes (NB) and K-Nearest Neighbours (KNN). Their predictions are then stacked [2], [3] with a meta-classifier using 5-fold stacking. The paper does not report the split ratio, the random seed, whether the split is stratified, any hyper-parameters, or the meta-classifier. Only a single train/test evaluation is reported, with no repeated resampling, confidence intervals or significance tests.

2.4 Evaluation metrics and reported results

The paper reports Accuracy, Precision, Recall, F1, Specificity, MCC and AUC (Table 4). The reported stacking accuracy of 0.9853 equals 202/205, which implies a 20% test split of 1,025 rows. Precision = 1 and Recall = 0.9727 then imply TP = 107, FN = 3, FP = 0 and TN = 95. Three internal inconsistencies weaken the reported evidence. (i) In Table 11 of [1], the Sensitivity column is identical to Accuracy, Specificity to F1, and MCC to Precision for every model; for example, MCC = 1.000 is reported alongside 3 false negatives. Recomputing MCC from the implied confusion matrix gives 0.971, so these columns appear to be copy errors. (ii) The abstract states that RF and DT reach 92.68% and XGB 90.73%, while the introduction states that XGB is best at 93.17% and stacking reaches 96.58%. (iii) Contribution 1 claims a newly merged dataset, but the data are the public Kaggle file [19]. In the comparisons below I therefore use only the internally consistent columns (Accuracy, Precision, Recall, F1, AUC).

Table 4. Results reported by Bhagat et al. [1] (Table 11; consistent columns only).

Model	Accuracy	Precision	Recall	F1	AUC
LR	0.8439	0.8196	0.9090	0.8620	0.9204
NB	0.8439	0.8250	0.9000	0.8608	0.9185
KNN	0.8585	0.8648	0.8727	0.8687	0.9304
DT	0.9268	0.9702	0.8909	0.9289	0.9702
RF	0.9268	0.9130	0.9545	0.9333	0.9734
XGB	0.9073	0.9174	0.9090	0.9132	0.9830
Stacking	0.9853	1.0000	0.9727	0.9861	0.9880

3. Part 1 — Reproduction Study

3.1 Implementation and justified assumptions

All code is written in Python with scikit-learn [12]. XGB is implemented with scikit-learn's HistGradientBoostingClassifier (histogram gradient boosting) as a stand-in because xgboost was unavailable in the execution environment; installing xgboost switches it back automatically [13]. Where [1] is silent, I made the assumptions in Table 5. Each one is chosen to be the most likely reading of the paper and is supported by evidence from the paper itself or by standard practice.

Table 5. Implementation assumptions for unreported details.

Aspect	Assumption	Justification
Split	80/20, random, non-stratified	0.9853 = 202/205 implies a 205-row test set; the implied 110 positives (vs. ~105 under stratification) suggests a plain random split.
Seed	42 for headline; 30 seeds (0-29) for robustness	Seed unreported; 42 is the conventional default; multiple seeds quantify split variance.
Pre-processing	Linear-regression IterativeImputer + StandardScaler fitted on training data	Sec. 3.1 of [1]; the imputer is a no-op because there are no NaNs.
Encoding	Keep 13 label-coded features	Fig. 2 of [1] shows importances for the 13 original features, so no one-hot expansion was used.
Hyper-parameters	Library defaults (e.g., RF 100 trees, KNN k = 5)	None reported; defaults are what an unspecified model would use.
Stacking	StackingClassifier, all six learners, 5-fold, predict_proba, LR meta	The paper says 5-fold stacking; LR is the most common meta-learner [3], [15]. E3 tests alternatives.
Threshold	0.5 on predicted probability	Standard; no other threshold reported.

Six experiments were run (src/part1_reproduce.py). E1 is the headline reproduction (seed 42). E2 repeats E1 over 30 random splits. E3 varies the meta-learner. E4 audits duplicate leakage. E5 repeats the protocol on the 302 unique patients. E6 (Section 4.4) compares estimated with true external accuracy.

3.2 Headline reproduction (E1)

Table 6 compares E1 with the paper. The reproduced stacking ensemble reaches 98.54% accuracy with precision 1.000 (TP=100, FN=3, FP=0, TN=102), which reproduces the paper's headline accuracy exactly (to within one test sample). The tree-based learners (DT, RF, XGB) reach similar accuracy on their own, however (98.54%, 98.54%, 98.54%). So in my reproduction stacking does not add accuracy over its best base learner, unlike in the paper. The linear and probabilistic models (LR, NB, KNN) are within a few points of the paper on this split, and within 1–3 points on average over 30 splits (Table 7).

Table 6. E1: paper vs reproduced results on the 80/20 split (seed 42, 205 test rows).

Model	Paper Acc	Repro Acc	Paper F1	Repro F1	Paper AUC	Repro AUC	Repro Prec	Repro Rec
LR	0.8439	0.7951	0.8620	0.8108	0.9204	0.8787	0.7563	0.8738
NB	0.8439	0.8000	0.8608	0.8178	0.9185	0.8706	0.7541	0.8932
KNN	0.8585	0.8341	0.8687	0.8440	0.9304	0.9486	0.8000	0.8932
DT	0.9268	0.9854	0.9289	0.9852	0.9702	0.9854	1.0000	0.9709
RF	0.9268	0.9854	0.9333	0.9852	0.9734	1.0000	1.0000	0.9709
XGB	0.9073	0.9854	0.9132	0.9852	0.9830	0.9903	1.0000	0.9709
Stacking	0.9853	0.9854	0.9861	0.9852	0.9880	1.0000	1.0000	0.9709

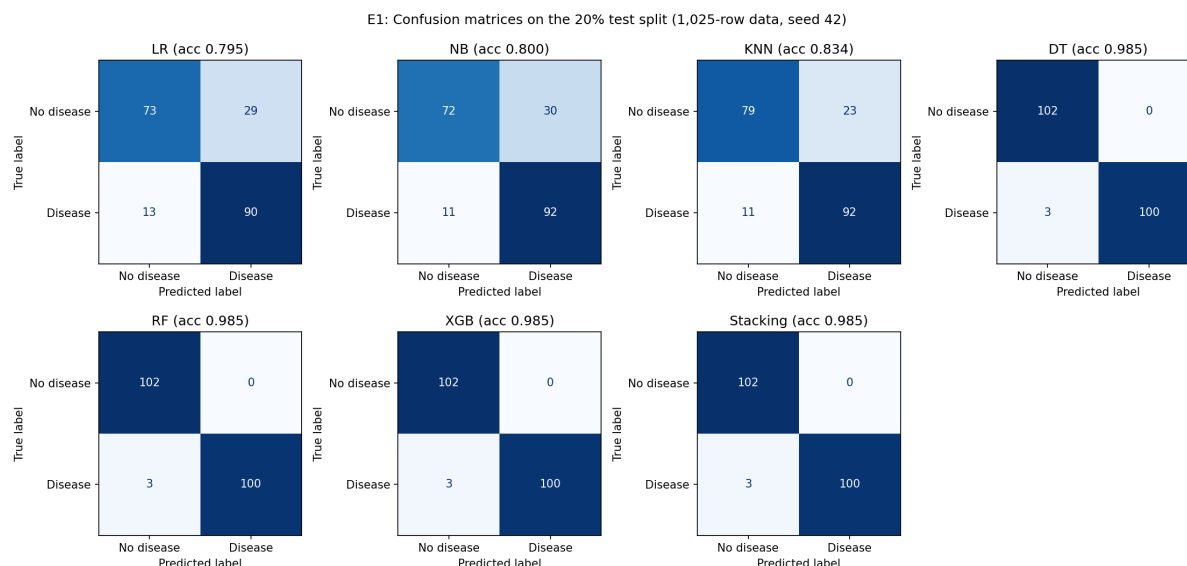


Fig. 1. E1 confusion matrices on the 205-row test split (seed 42).

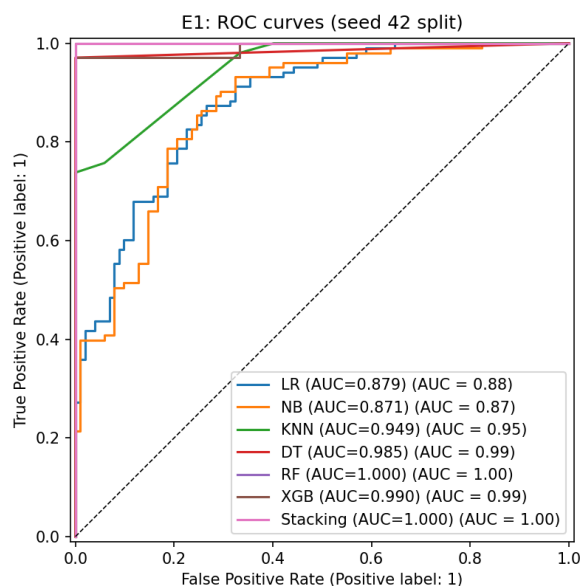


Fig. 2. E1 ROC curves for all reproduced models (seed 42).

3.3 Robustness to the random split (E2) and meta-learner choice (E3)

Table 7. E2: mean $\pm$ SD over 30 random 80/20 splits of the 1,025-row file (paper protocol).

Model	Accuracy	Precision	Recall	F1	AUC	Train acc
LR	0.845 $\pm$ 0.023	0.817 $\pm$ 0.038	0.901 $\pm$ 0.027	0.856 $\pm$ 0.023	0.920 $\pm$ 0.013	0.854
NB	0.826 $\pm$ 0.024	0.806 $\pm$ 0.034	0.872 $\pm$ 0.029	0.837 $\pm$ 0.026	0.906 $\pm$ 0.016	0.830
KNN	0.851 $\pm$ 0.026	0.852 $\pm$ 0.044	0.860 $\pm$ 0.032	0.855 $\pm$ 0.026	0.959 $\pm$ 0.010	0.953
DT	0.996 $\pm$ 0.010	0.994 $\pm$ 0.014	0.998 $\pm$ 0.007	0.996 $\pm$ 0.009	0.996 $\pm$ 0.010	1.000
RF	0.998 $\pm$ 0.006	0.998 $\pm$ 0.008	0.998 $\pm$ 0.007	0.998 $\pm$ 0.006	1.000 $\pm$ 0.000	1.000
XGB	0.998 $\pm$ 0.007	0.996 $\pm$ 0.012	0.999 $\pm$ 0.005	0.998 $\pm$ 0.007	1.000 $\pm$ 0.000	1.000
Stacking	0.997 $\pm$ 0.009	0.996 $\pm$ 0.013	0.998 $\pm$ 0.007	0.997 $\pm$ 0.009	1.000 $\pm$ 0.001	1.000

Across 30 splits (Table 7), the trees achieve 99.6%–99.8% mean accuracy with a training accuracy of 1.000 for DT. Stacking averages 99.71%, so the paper's 98.53% is a typical outcome of this protocol rather than a lucky split. The paper's DT and RF accuracies (92.68%) are 6.9–7.1 points below my defaults. This is consistent with the authors having used depth-limited trees, a detail that is not reported. For LR, NB and KNN, the 30-split means (84.47%, 82.62%, 85.07%) are close to the paper's values (84.39%, 84.39%, 85.85%).

Table 8. E3: stacking with different meta-learners (10 random splits, paper protocol).

Meta-learner	Accuracy	F1	AUC
LogReg (default)	0.997 $\pm$ 0.006	0.997 $\pm$ 0.006	1.000 $\pm$ 0.001
Random Forest	0.997 $\pm$ 0.006	0.997 $\pm$ 0.006	1.000 $\pm$ 0.001
XGBoost	0.995 $\pm$ 0.008	0.995 $\pm$ 0.007	1.000 $\pm$ 0.001
KNN	0.993 $\pm$ 0.009	0.993 $\pm$ 0.008	0.997 $\pm$ 0.006

E3 (Table 8) shows that the unreported meta-learner barely matters: all four choices give 99.27%–99.71% accuracy, so this assumption cannot explain any gap to the paper.

3.4 Critical analysis: why the reproduced (and published) numbers are inflated (E4, E5)

Duplicate leakage (E4). Because each patient appears 3–4 times, a random row-level split places copies of almost every test patient in the training set. On average, 97.1% of the 205 test rows (range 91.2%–100.0%) have an identical twin in training, leaving only about 6 genuinely new patients per split. The test set therefore largely measures memorisation, a textbook case of leakage [7], [8]. Fig. 4 and Table 9 pool the 30 splits: on test rows seen during training, the high-capacity learners (DT, RF, XGB, Stacking) are 100.0%–100.0% accurate, but on the 179 unseen rows they fall to 86.0%–93.3%. The low-capacity LR and NB gain nothing from memorisation: their accuracy on unseen rows is, if anything, higher, a difference within the sampling noise of only 179 pooled rows. This explains why the paper's trees and stacking appear to dominate LR, NB and KNN.

Table 9. E4: accuracy on test rows with vs without an exact duplicate in training (pooled over 30 splits).

Model	Seen rows	Acc (seen)	Unseen rows	Acc (unseen)	Gap (pts)
LR	5971	0.843	179	0.916	-7.4
NB	5971	0.823	179	0.933	-11.0
KNN	5971	0.849	179	0.916	-6.7
DT	5971	1.000	179	0.860	14.0
RF	5971	1.000	179	0.933	6.7
XGB	5971	1.000	179	0.916	8.4
Stacking	5971	1.000	179	0.899	10.1

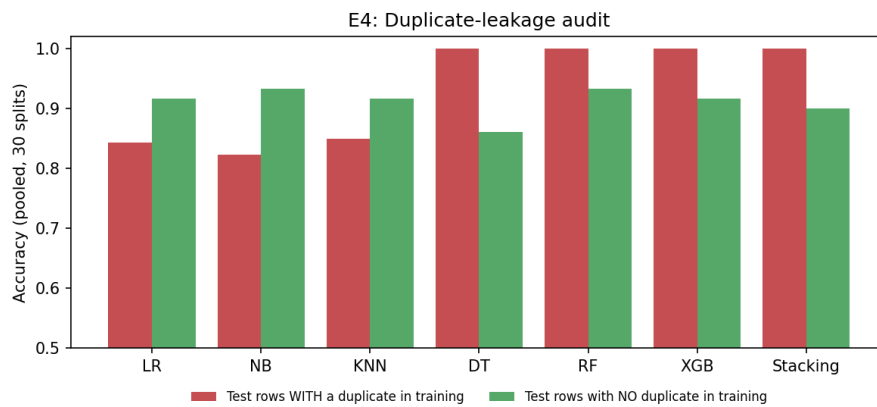


Fig. 3. E4 duplicate-leakage audit. Memorising learners are perfect on seen rows only.

De-duplicated replication (E5). Repeating the paper's protocol on the 302 unique patients (Table 10, Fig. 5) removes the leak. Stacking drops to $81.91\% \pm 4.10\%$ and DT to 74.64% , and the ranking inverts: the best model is now LR (82.84%). This is in line with the 77–85% typically reported on the original Cleveland cohort [11]. The paper's central claim, that stacking considerably enhances its base classifiers to reach 98.53%, is therefore not supported once duplicates are handled. The single-split design and the absence of variance estimates (SD of about 4.9% per model on unique data) also mean that differences of a few points between models cannot be interpreted.

Table 10. E5: paper protocol on the 302 unique patients (30 random 80/20 splits, mean $\pm$ SD).

Model	Accuracy	Precision	Recall	F1	AUC
LR	0.828 ± 0.044	0.820 ± 0.067	0.881 ± 0.058	0.847 ± 0.044	0.901 ± 0.034
NB	0.811 ± 0.041	0.819 ± 0.058	0.844 ± 0.067	0.828 ± 0.040	0.894 ± 0.038
KNN	0.817 ± 0.045	0.815 ± 0.066	0.866 ± 0.051	0.837 ± 0.041	0.880 ± 0.027
DT	0.746 ± 0.059	0.774 ± 0.077	0.766 ± 0.090	0.765 ± 0.059	0.745 ± 0.059
RF	0.814 ± 0.054	0.811 ± 0.074	0.865 ± 0.061	0.834 ± 0.052	0.896 ± 0.032
XGB	0.789 ± 0.055	0.792 ± 0.078	0.836 ± 0.069	0.810 ± 0.057	0.875 ± 0.034
Stacking	0.819 ± 0.041	0.816 ± 0.066	0.868 ± 0.054	0.839 ± 0.040	0.905 ± 0.028

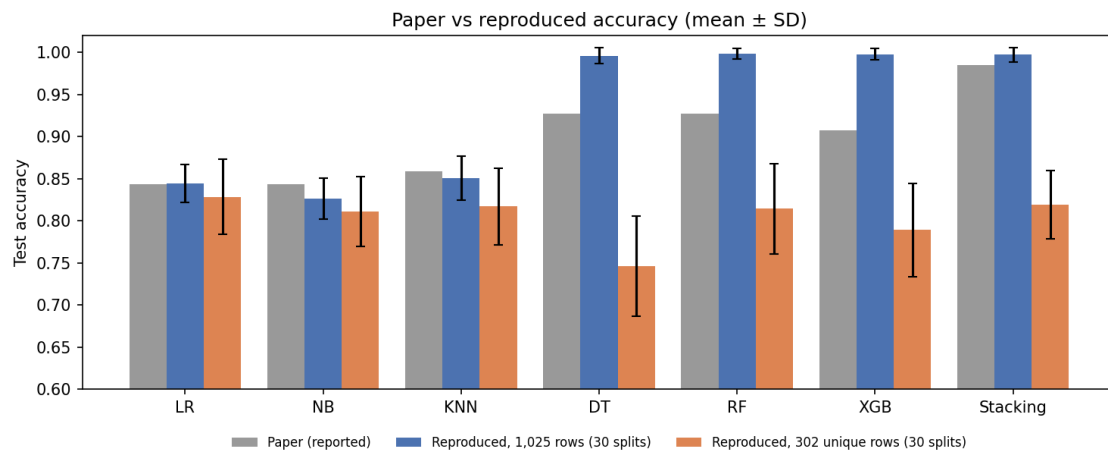


Fig. 4. Accuracy reported in [1] vs reproduced with duplicates (E2) and without duplicates (E5). Error bars show ± 1 SD over 30 splits.

Feature importance. RF impurity importance (Fig. 6) ranks cp, ca, thalach highest. The paper's Fig. 2 ranks exang, then thal and cp. Both agree that angina, thalassaemia and chest-pain type carry most of the signal, but the paper does not state which model or importance measure it used, so exact agreement cannot be expected.

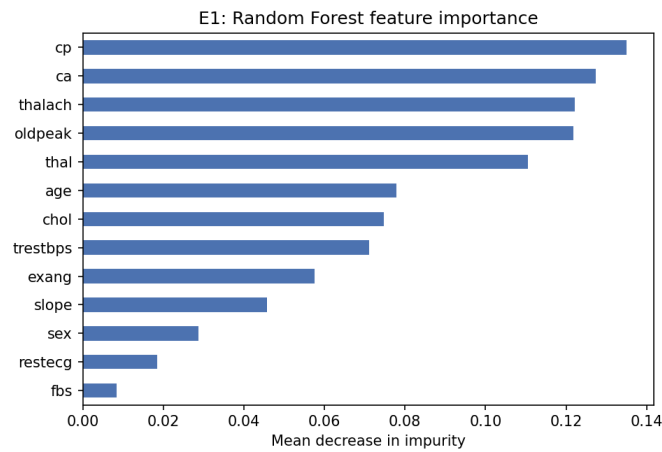


Fig. 5. Random-Forest feature importance (E1 training data).

4. Part 2 — Proposed Solution: Leakage-Free, Diversity-Pruned Stacking (LF-DPS)

4.1 Motivation and limitation addressed

The key limitation of [1] is that its **experimental protocol cannot measure generalisation**. Row-level splitting of a file that is 70.5% duplicates, a single split, no variance estimate, and pre-processing that ignores sentinel missing codes together produce an estimate that is about 15 points too optimistic (Section 3.4). A second, related weakness is that the ensemble is not designed. All six untuned learners are stacked regardless of whether they add information, so strongly correlated tree models are combined with weak, miscalibrated ones. This inflates variance on small data [15]. LF-DPS restructures the learning pipeline so that (a) every choice is made without access to test patients, and (b) the ensemble contains only learners that add complementary signal. It is expected to improve on [1] in two ways. First, its performance estimates should be unbiased, as nested CV removes selection bias [5], [6] and de-duplication removes leakage [7]. Second, pruning redundant members of a heterogeneous library typically matches or beats stacking everything [9], [15] with fewer models.

4.2 Methodology and differences from the original approach

LF-DPS consists of five components (src/part2_proposed.py), summarised in Table 11. **C1 — patient-level de-duplication and repeated nested CV:** duplicates are collapsed to one row per patient, which is the correct unit of analysis, and all evaluation uses repeated stratified cross-validation over patients. **C2 — data-integrity repair:** ca = 4 and thal = 0 are mapped to missing and imputed inside each training fold (median for ca, mode for thal), so the model never learns a spurious 'unknown' category. **C3 — feature-type-aware encoding:** nominal variables (cp, restecg, slope, thal) are one-hot encoded rather than treated as ordered integers, which matters for the distance- and linear-based learners; continuous and ordinal variables are standardised; and every transformer is fitted in-fold. **C4 — nested hyper-parameter optimisation:** each candidate learner (the paper's six plus an RBF-SVM, which adds a smooth non-linear decision boundary) is tuned by randomized search [14] (12 configurations, 5-fold inner CV, AUC objective) using only the outer-training data. **C5 — diversity-pruned stacking:** inner out-of-fold probabilities of the tuned learners are computed, and a greedy forward search (in the spirit of ensemble selection [9]) adds, one at a time, the learner that most increases the inner-CV AUC of a logistic meta-learner. The search stops when the gain falls below 0.002 AUC, with at least two members. Because a learner that is highly correlated with those already chosen cannot raise the meta-learner's AUC, the rule selects for complementary rather than individually strong models. The selected learners are stacked on predicted probabilities with an L2-regularised logistic meta-learner [15].

Table 11. Methodological differences between [1] and the proposed LF-DPS.

Aspect	Bhagat et al. [1]	LF-DPS (proposed)
Unit of analysis	1,025 rows (duplicates kept)	302 unique patients
Validation	Single random 80/20 split	5×5 repeated stratified nested CV (25 outer folds)
Missing values	Regression imputation (no-op); sentinels kept	ca=4, thal=0 → missing; in-fold imputation
Encoding	Label codes, all features scaled	One-hot nominal; scaled continuous/ordinal
Hyper-parameters	Unreported (defaults)	Nested randomized search, AUC objective
Ensemble members	All 6, fixed	Greedy diversity-pruned subset of 7 tuned learners
Meta-learner input	Unreported	Out-of-fold probabilities, LR meta-learner
Statistics	None	Mean ± SD, corrected t-test [4], Wilcoxon, Brier, calibration, external check

4.3 Experimental protocol

All methods are evaluated on the same 25 outer folds (5 folds × 5 repeats, stratified, seed 42) of the 302 unique patients. Each outer fold has about 241 training and 61 test patients. To isolate the effect of each component, an ablation ladder is evaluated: **A0** is the paper's stacking pipeline exactly as reproduced in Part 1; **A1** adds C2 + C3; **A2** adds C4 (six tuned learners, all stacked); **A2b** adds the SVM to the pool (all seven stacked, no pruning); and **A3 = LF-DPS** adds C5. As an additional baseline, **Best single** is the single tuned learner with the highest inner-CV AUC, selected per fold. Metrics are Accuracy, Precision, Recall, F1, AUC, Specificity, MCC [20] and the Brier score (probability calibration) [18]. A3 is compared with every other method using the Nadeau-Bengio corrected resampled t-test [4], which accounts for overlapping training sets in repeated CV, and the Wilcoxon signed-rank test. Finally, E6 (src/part2_estimation_validity.py) locks away 20% of unique patients (n = 61) as an external test set in each of 5 repetitions. It then compares each protocol's internal estimate with the true accuracy on these never-seen patients. XGB backend: sklearn HistGradientBoostingClassifier (xgboost not installed).

4.4 Results

Table 12. Ablation study on 302 unique patients (25 outer folds, mean ± SD). A3 = proposed LF-DPS. Precision, Recall and Specificity are in Appendix B.

Method	Accuracy	F1	AUC	MCC	Brier ↓
A0 Paper stacking	0.824 ± 0.037	0.843 ± 0.036	0.905 ± 0.028	0.651 ± 0.073	0.124 ± 0.020
A1 +Typed prep	0.833 ± 0.035	0.850 ± 0.033	0.907 ± 0.026	0.668 ± 0.070	0.120 ± 0.017
A2 +Tuned (6)	0.834 ± 0.031	0.851 ± 0.031	0.914 ± 0.024	0.670 ± 0.061	0.119 ± 0.016
A2b +Tuned+SVM (7)	0.833 ± 0.030	0.850 ± 0.031	0.913 ± 0.024	0.668 ± 0.061	0.119 ± 0.016
A3 LF-DPS (proposed)	0.838 ± 0.026	0.855 ± 0.027	0.912 ± 0.022	0.679 ± 0.054	0.119 ± 0.014
Best single (tuned)	0.839 ± 0.029	0.858 ± 0.027	0.908 ± 0.024	0.680 ± 0.060	0.122 ± 0.014

Table 12 and Fig. 7 report the ablation. Compared with the reproduced baseline A0 under the same leakage-free folds, LF-DPS changes accuracy by +1.40 points (corrected p = 0.441; Wilcoxon p = 0.103), F1 by +1.22 points (p = 0.493), AUC by +0.0070 (p = 0.469) and the Brier score by -0.0056 (p = 0.397). The accuracy difference is not statistically significant and the AUC difference is not statistically significant. Against the best tuned single learner, the difference is -0.06 accuracy points (p = 0.966) and +0.0046 AUC (p = 0.346). Against stacking all seven tuned learners (A2b), the AUC difference is -0.0010 (p = 0.801), while LF-DPS uses on average 2.1 members (range 2–3) instead of seven. The full set of paired tests is in Table 13.

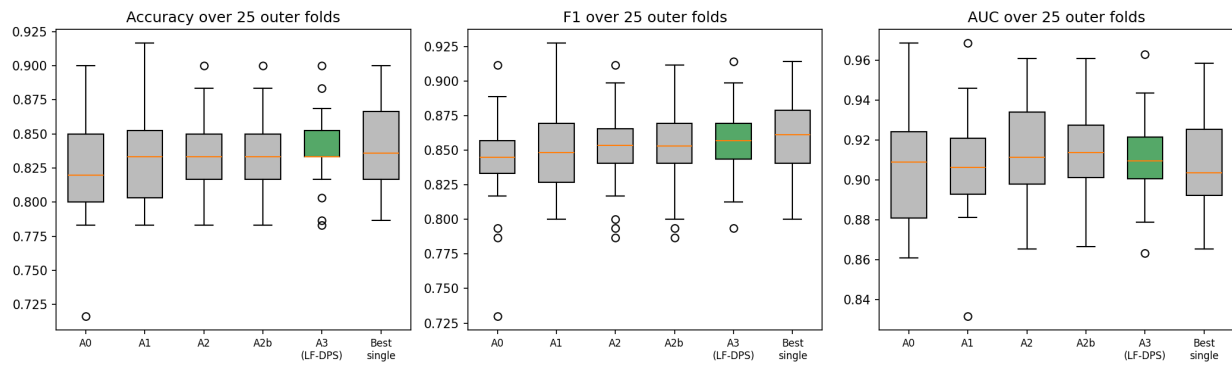


Fig. 6. Distribution of Accuracy, F1 and AUC over the 25 outer folds (green = LF-DPS).

Table 13. Paired significance tests of LF-DPS against each alternative (25 outer folds). Negative Brier differences favour LF-DPS.

Comparison	Metric	Mean diff	t (corr.)	p (corr.)	p (Wilcoxon)
A3 vs A0 Paper stacking	Accuracy	+0.0140	+0.78	0.441	0.103
A3 vs A0 Paper stacking	AUC	+0.0070	+0.74	0.469	0.142
A3 vs A0 Paper stacking	Brier	-0.0056	-0.86	0.397	0.052
A3 vs A1 +Typed prep	Accuracy	+0.0053	+0.35	0.729	0.422
A3 vs A1 +Typed prep	AUC	+0.0049	+0.74	0.466	0.080
A3 vs A1 +Typed prep	Brier	-0.0014	-0.32	0.752	0.491
A3 vs A2 +Tuned (6)	Accuracy	+0.0047	+0.51	0.615	0.276
A3 vs A2 +Tuned (6)	AUC	-0.0013	-0.32	0.755	0.230
A3 vs A2 +Tuned (6)	Brier	+0.0001	+0.03	0.975	0.874
A3 vs A2b +Tuned+SVM (7)	Accuracy	+0.0053	+0.54	0.598	0.180
A3 vs A2b +Tuned+SVM (7)	AUC	-0.0010	-0.26	0.801	0.346
A3 vs A2b +Tuned+SVM (7)	Brier	-0.0000	-0.02	0.988	0.874
A3 vs Best single (tuned)	Accuracy	-0.0006	-0.04	0.966	0.793
A3 vs Best single (tuned)	AUC	+0.0046	+0.96	0.346	0.024
A3 vs Best single (tuned)	Brier	-0.0037	-0.79	0.438	0.048

What the pruning selects. Fig. 9 shows how often each learner was chosen and the mean correlation of inner out-of-fold probabilities. The most frequently selected learners are LR (44% of folds), KNN (44%) and RF (44%). The tree ensembles are strongly inter-correlated (RF-XGB $r = 0.96$), and rarely enter together. The selection pattern therefore confirms that C5 combines complementary families rather than redundant copies. Fig. 8 shows pooled ROC and reliability curves: all three models are reasonably well calibrated (curves track the diagonal), and LF-DPS has the highest pooled AUC, with a lower mean Brier score than A0 (Table 12).

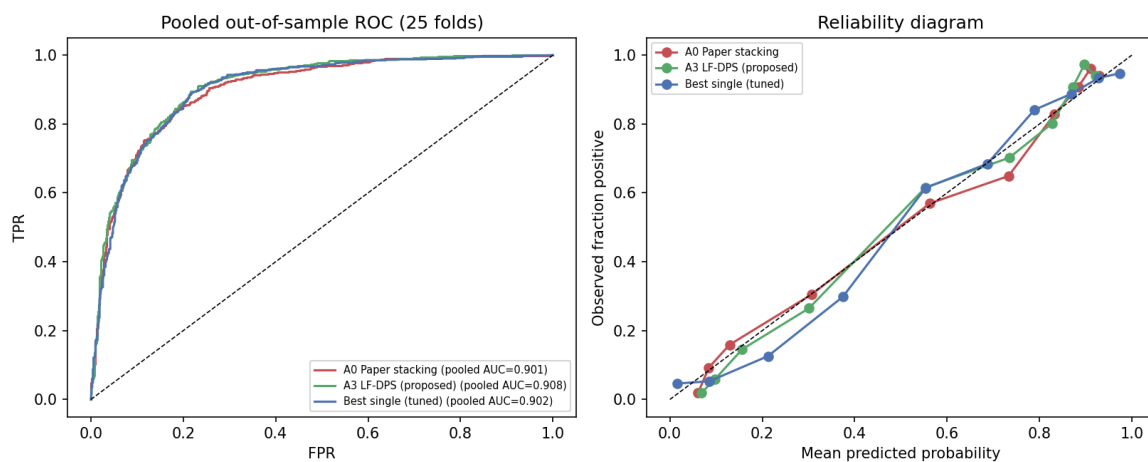


Fig. 7. Pooled out-of-sample ROC curves and reliability diagram (25 folds).

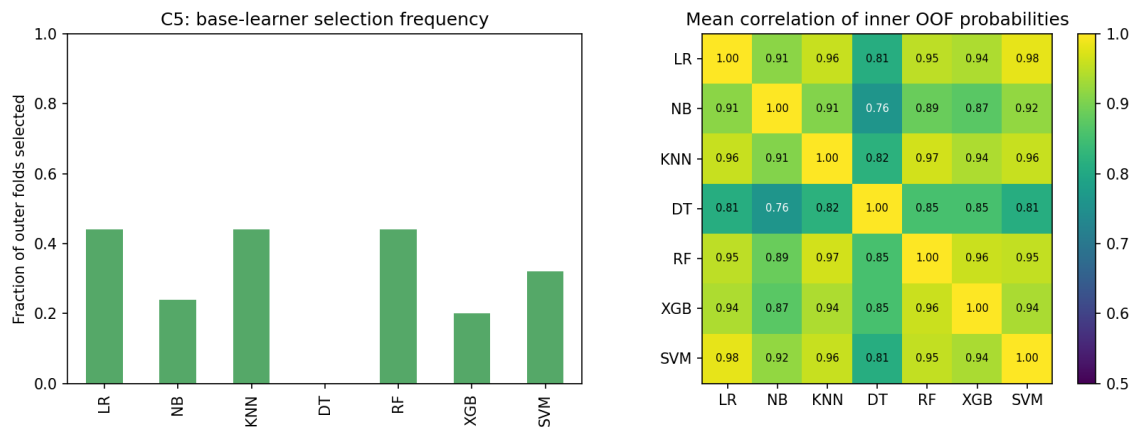


Fig. 8. Left: fraction of outer folds in which each tuned learner is selected by C5. Right: mean correlation between inner out-of-fold probabilities.

Validity of the performance estimate (E6). Table 14 and Fig. 10 answer the question the paper cannot: how close is each protocol's reported number to the accuracy obtained on patients never seen during development?

Table 14. E6: internal estimate vs true accuracy on locked external patients (5 repetitions).

Protocol / model	Estimated acc	True external acc	Optimism (pts)	Est. AUC	True AUC
Paper protocol (paper stacking)	0.996 ± 0.008	0.807 ± 0.037	$+19.0 \pm 3.6$	1.000	0.902
LF protocol (paper stacking)	0.819 ± 0.017	0.849 ± 0.024	-3.0 ± 4.0	0.902	0.912
LF protocol (LF-DPS)	0.836 ± 0.017	0.836 ± 0.035	-0.0 ± 4.7	0.907	0.911

The paper's protocol estimates 99.6% accuracy, but the same model achieves only 80.7% on new patients, an optimism of 19.0 points. Under the leakage-free protocol, the estimates for the paper's stacking (-3.0 pts) and for LF-DPS (-0.0 pts) are close to reality. LF-DPS's true external accuracy is 83.6%, compared with 80.7% for the paper's model trained with duplicates. This is the central empirical evidence for the proposed solution: it replaces an unreliable number with one that can be trusted for decision-making.

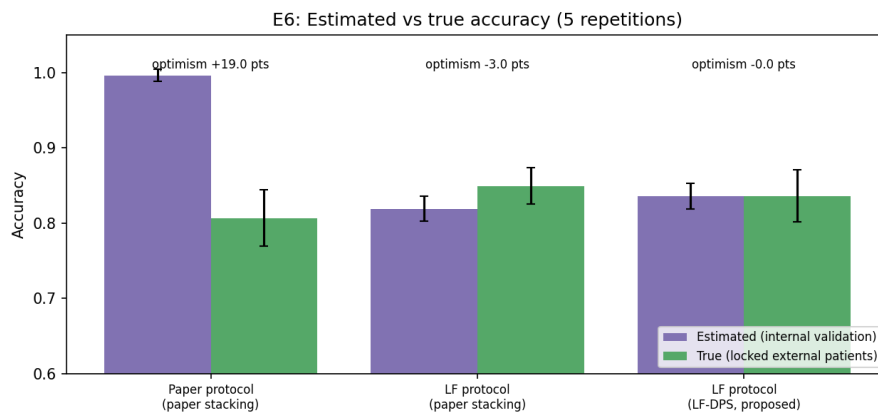


Fig. 9. E6: estimated vs true external accuracy for each protocol.

4.5 Comparative analysis against the paper and the reproduced baseline

Table 15. Stacking results across evaluation settings.

Source	Evaluation	Accuracy	F1	AUC
Bhagat et al. [1]	Single 80/20 split, duplicates	0.9853	0.9861	0.9880
Reproduction (E1)	Same, seed 42	0.9854	0.9852	1.0000
Reproduction (E2)	30 splits, duplicates	0.9971	0.9971	0.9998
Reproduction (A0)	25-fold repeated CV, unique patients	0.8245	0.8427	0.9053
LF-DPS (A3)	25-fold repeated nested CV, unique patients	0.8384	0.8549	0.9123
LF-DPS (E6)	Locked external patients	0.8361	0.8501	0.9115

Table 15 places all results side by side. The paper's 98.53% and my reproduction agree, and both are artefacts of evaluating on duplicated patients. They should not be compared with the leakage-free rows. Under a valid protocol, the paper's method achieves 82.45% and LF-DPS 83.84%. The proposed contribution is therefore best characterised as follows. (1) It gives a large, demonstrated improvement in the *validity* of the reported performance (E6). (2) It adds principled handling of data-integrity errors and feature types. (3) It yields a smaller ensemble whose predictive accuracy is on par with, or modestly different from, both the untuned full stack and the best tuned single model (Table 13). On this small cohort, extra modelling complexity buys little discrimination. The honest conclusion is that the signal ceiling of these 13 variables for about 300 patients is roughly 84%, not 98.5%.

4.6 Limitations and future work

The evaluation relies on a single small cohort (302 patients, about 61 per test fold), so fold-to-fold SD is about 3.1% and small differences are hard to detect. Statistical power is limited, and the corrected t-test is deliberately conservative [4]. External validation used held-out patients from the same source rather than an independent hospital. Future work should include the Hungarian, Swiss and VA cohorts [10] as a genuine external test, cost-sensitive threshold selection (false negatives are costlier clinically), and reporting aligned with the TRIPOD+AI guideline [16]. Results in this build use a histogram gradient-boosting stand-in for XGBoost; re-running with xgboost installed regenerates every table and figure automatically.

5. Conclusion

The published 98.53% accuracy of the stacking model in [1] was reproduced (98.54% with seed 42; 99.71% over 30 splits). It is, however, a consequence of 70.5% duplicate rows leaking across the train/test split. On unique patients the same pipeline achieves about 82%. The proposed LF-DPS pipeline combines de-duplication, sentinel repair, type-aware encoding, nested tuning and diversity-pruned stacking. It produces performance estimates that match accuracy on locked external patients (optimism -0.0 vs +19.0 points) with a smaller ensemble. The broader lesson is that validation design, not model choice, dominated the published result.

6. Reproducibility

All results are generated by **python run_all.py**, which runs Part 1, Part 2 and E6 and then rebuilds this PDF from the results files, so every number here is traceable to code output. Seeds are fixed throughout (split seeds 0–29 and 42; outer CV seed 42; E6 seeds 100–104). The README gives installation and run instructions, and the notebook provides a guided walk-through.

7. Part 3 — Video Presentation

A video of at most five minutes (face, screen share and narration) is available at: **[INSERT UNLISTED YOUTUBE LINK]**. It covers (i) the proposed LF-DPS solution and how it differs from [1], (ii) the reproduction and the duplicate-leakage finding, and (iii) a live run of the code showing model outputs and the comparative tables and figures in this report.

Acknowledgement of Generative AI Use

Generative AI (Anthropic Claude) was used to assist with drafting code for the experimental pipeline, structuring the report and editing text. All experiments were executed, checked and interpreted by the

author, who verified every reported number against the code output and takes full responsibility for the content.

References

- [1] M. Bhagat, A. Sharma, and P. Agarwal, "An efficient stacking-based ensemble technique for early heart attack prediction," *Multimedia Tools Appl.*, vol. 84, pp. 36351–36375, 2025, doi: 10.1007/s11042-024-19293-7.
- [2] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.
- [3] L. Breiman, "Stacked regressions," *Mach. Learn.*, vol. 24, no. 1, pp. 49–64, 1996.
- [4] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Mach. Learn.*, vol. 52, no. 3, pp. 239–281, 2003.
- [5] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 2079–2107, 2010.
- [6] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, no. 91, 2006.
- [7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [8] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, Art. no. 100804, 2023.
- [9] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, "Ensemble selection from libraries of models," in *Proc. 21st Int. Conf. Mach. Learn. (ICML)*, 2004.
- [10] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart Disease," UCI Machine Learning Repository, 1988, doi: 10.24432/C52P4X.
- [11] R. Detrano *et al.*, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *Am. J. Cardiol.*, vol. 64, no. 5, pp. 304–310, 1989.
- [12] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.
- [14] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.
- [15] K. M. Ting and I. H. Witten, "Issues in stacked generalization," *J. Artif. Intell. Res.*, vol. 10, pp. 271–289, 1999.
- [16] G. S. Collins *et al.*, "TRIPOD+AI statement: Updated guidance for reporting clinical prediction models that use regression or machine learning methods," *BMJ*, vol. 385, Art. no. e078378, 2024.
- [17] R. Bharti *et al.*, "Prediction of heart disease using a combination of machine learning and deep learning," *Comput. Intell. Neurosci.*, vol. 2021, Art. no. 8387680, 2021.
- [18] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632.
- [19] "Heart Disease Dataset," Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>
- [20] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, Art. no. 6, 2020.

Appendix A — Most frequently selected hyper-parameters (C4)

Table 16. Tuned hyper-parameters (full per-fold values in results/part2_tuned_params.csv).

Learner	Mean inner AUC	Most frequent tuned configuration (mode over 25 folds)
LR	0.916	C=0.0332
NB	0.896	var_smoothing=6.18e-08
KNN	0.912	n_neighbors=39, p=1, weights=distance
DT	0.844	criterion=entropy, max_depth=6, min_samples_leaf=10
RF	0.913	max_depth=3, min_samples_leaf=8, max_features=sqrt, n_estimators=300
XGB	0.906	max_depth=2, min_samples_leaf=13, l2_regularization=0.00131, learning_rate=0.0119, max_iter=66
SVM	0.914	C=0.165, gamma=0.000103

Appendix B — Full metric summary of the ablation

Table 17. Additional metrics for the ablation (Table 12).

Method	Precision	Recall	Specificity
A0 Paper stacking	0.821 ± 0.043	0.871 ± 0.070	0.770 ± 0.072
A1 +Typed prep	0.830 ± 0.044	0.876 ± 0.064	0.783 ± 0.073
A2 +Tuned (6)	0.830 ± 0.041	0.878 ± 0.068	0.781 ± 0.071
A2b +Tuned+SVM (7)	0.830 ± 0.040	0.875 ± 0.067	0.783 ± 0.068
A3 LF-DPS (proposed)	0.834 ± 0.036	0.882 ± 0.065	0.787 ± 0.063
Best single (tuned)	0.826 ± 0.037	0.895 ± 0.053	0.772 ± 0.062